

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e de Informática

Arthur Victor L. de M. Alves¹
Iago Verciano Romeiro²

Trabalho Prático — Pipeline Escalar e Superescalar

Sumário

1	Introdução	1
2	Fundamentação Teórica	1
2.1	O Pipeline Escalar Clássico de 5 Estágios	1
2.2	Conflitos de Pipeline (<i>Hazards</i>)	2
2.2.1	Hazards Estruturais	2
2.2.2	Hazards de Dados	2
2.2.3	Hazards de Controle	3
2.3	Técnicas de Mitigação: <i>Forwarding</i> e Predição de Desvios	3
2.3.1	Adiantamento de Dados (<i>Forwarding / Bypass</i>)	3
2.3.2	Predição de Desvios Estática e Dinâmica	3
2.4	Arquiteturas Superescalares e Execução Fora de Ordem	4
2.5	O Algoritmo de Tomasulo	4
2.6	Renomeação de Registradores via Tabela de Apelidos (<i>RAT</i>)	5
2.7	Buffer de Reordenação (<i>Reorder Buffer — ROB</i>)	5
3	Metodologia Experimental	6
4	Resultados e Análise Quantitativa	7
4.1	Visualização de Desempenho	7
5	Análise Detalhada por Sequência (Pipeline Escalar)	7
5.1	Sequência S2 — Loop (Soma de Array)	7
5.1.1	Questões Práticas (S2)	7
5.2	Sequência S3 — Load-Use	10
5.2.1	Questões Práticas (S3)	10
5.3	Sequência S4 — WAR e WAW (Anti-dependência e Dependência de Saída)	11
5.3.1	Questões Práticas (S4)	11
5.4	Sequência S5 — Desvios Condicionais Mistos	11
5.4.1	Questões Práticas (S5)	11
6	Análise Avançada e Respostas Teóricas (Roteiro A.4)	12
6.1	Cálculo de Speedup	12
6.2	Impacto na Equação de Tempo Fundamental	13
6.3	Discussão Conceitual: Preditores de Desvio em Loops	13
6.4	Mapeamento de Hazards Dominantes	13
7	Discussão Geral e Extensão Superescalar	13
8	Pipeline Superescalar: Tomasulo, Renomeação e Execução Fora de Ordem	14
8.1	Algoritmo de Tomasulo — Simulação Manual	14
8.1.1	Sequência T1: Dependências RAW em Cadeia	14
8.1.2	Sequência T2: Eliminação de WAR e WAW	15
8.2	Renomeação de Registradores — Análise Manual	16
8.2.1	1. Identificação de Dependências na Sequência S4	16
8.2.2	2. Mapeamento Passo a Passo via Register Alias Table (RAT)	16
8.2.3	3. Dependências Remanescentes e ILP Máximo Teórico	17

8.2.4	4. Comparação Crítica de Modelos de Exploração de ILP	17
8.3	Simulação Superescalar e B.4 Análise Requerida	17
8.3.1	Métricas de Ocupação e Flushes	18
8.3.2	Aplicação da Lei de Amdahl ao ILP	18
8.3.3	Discussão de Gargalos Dominantes e Comportamento do ROB . . .	19
9	Análise Integradora — Do Escalar ao Superescalar	20
9.1	1. Curva de Evolução do IPC para a Sequência S2 (Loop)	20
9.2	2. Cálculo do IPC Efetivo com Penalidade de Misprediction	21
9.3	3. O Paradoxo da Dependência de Memória em Processadores OoO	21
9.4	4. Comparação Crítica de Abordagens de Gerenciamento de Dependências	23
10	Conclusão Geral	23
11	Apêndices	25
11.1	Apêndice I — Sequências de Instruções Utilizadas	25
11.2	Apêndice II — Dados Brutos e Repositório Git	25

Resumo

Este trabalho analisa o comportamento de pipelines escalares e superescalares utilizando o simulador RIPES baseado na arquitetura RISC-V. Foram avaliados hazards de dados e controle, técnicas de forwarding, predição de desvios, escalonamento estático, algoritmo de Tomasulo, renomeação de registradores e execução fora de ordem. Os experimentos mediram CPI, IPC, stalls e speedup em diferentes configurações arquiteturais. Os resultados demonstraram ganhos significativos de desempenho com forwarding e predição dinâmica de desvios, além do aumento de ILP em arquiteturas superescalares com ROB e execução OoO. Conclui-se que técnicas modernas de controle dinâmico são essenciais para maximizar o paralelismo e reduzir penalidades de hazards em processadores atuais.

1 Introdução

Este relatório apresenta a análise experimental do comportamento de pipelines escalares e superescalares sob diferentes condições de conflitos (*hazards*) de dados e de controle. A simulação foi executada utilizando o simulador RIPES baseado no conjunto de instruções RISC-V, permitindo observar ciclos de *clock*, stalls, adiantamento (*forwarding*) e o impacto de diferentes estratégias de predição de desvios.

O objetivo deste estudo é quantificar o impacto dos *hazards* através do cálculo do CPI (*Cycles Per Instruction*) em cinco sequências de código estruturadas (S1 a S5), sob diferentes configurações arquiteturais.

2 Fundamentação Teórica

2.1 O Pipeline Escalar Clássico de 5 Estágios

O conceito de paralelismo em nível de instrução (*Instruction-Level Parallelism* — ILP) fundamenta-se na sobreposição temporal de diferentes etapas da execução de instruções[cite: 15, 17]. Em uma arquitetura escalar RISC clássica, o fluxo de processamento é segmentado em cinco estágios independentes e síncronos[cite: 39]:

1. **Busca da Instrução (IF — *Instruction Fetch*):** O endereço contido no Contador de Programa (*Program Counter* — PC) é enviado à memória de instruções para a extração do código binário da instrução. O PC é subsequentemente incrementado para apontar para a instrução sequencial seguinte.
2. **Decodificação e Leitura de Registradores (ID — *Instruction Decode*):** O código de operação (*opcode*) e os campos de operandos são decodificados pela unidade de controle. Simultaneamente, os dados dos registradores de origem são lidos do Banco de Registradores.
3. **Execução e Cálculo de Endereço Eficaz (EX — *Execution*):** A Unidade Lógica e Algorítmica (ULA) realiza a operação aritmética ou lógica especificada. Em instruções de acesso à memória (*loads* e *stores*), a ULA calcula o endereço de memória eficaz somando o registrador de base ao operando imediato.
4. **Acesso à Memória (MEM — *Memory Access*):** Caso a instrução seja de leitura (*load*), os dados são extraídos da memória de dados no endereço computado no

estágio EX. Caso seja de escrita (*store*), os dados previamente lidos do banco de registradores são gravados na memória. Instruções aritméticas puras apenas propagam seus resultados.

5. **Escrita no Banco de Registradores (WB — *Write-Back*):** O resultado final, seja ele proveniente da ULA ou da memória de dados, é efetivamente consolidado e gravado de volta no registrador de destino do Banco de Registradores.

Em condições ideais, o pipeline permite que uma nova instrução conclua sua execução a cada ciclo de clock, atingindo um Cycles Per Instruction (CPI) ideal de 1,0 (ou um Instructions Per Cycle — IPC de 1,0).

2.2 Conflitos de Pipeline (*Hazards*)

Na prática microarquitetural, a continuidade do fluxo de instruções é frequentemente interrompida por situações conhecidas como *hazards* (conflitos), as quais impedem a execução da próxima instrução no ciclo de clock subsequente. Os *hazards* dividem-se em três categorias estritas[cite: 28, 178]:

2.2.1 Hazards Estruturais

Ocorrem quando duas ou mais instruções em estágios distintos do pipeline tentam acessar simultaneamente o mesmo recurso físico de hardware. Um exemplo clássico se manifesta quando o processador possui uma memória unificada para instruções e dados, gerando um conflito estrutural quando uma instrução no estágio IF tenta buscar um código enquanto uma instrução anterior no estágio MEM tenta ler ou escrever dados. A mitigação padrão envolve a separação física em caches de primeiro nível distintas (Arquitetura Harvard: Cache L1I e Cache L1D).

2.2.2 Hazards de Dados

Manifestam-se quando o alinhamento temporal do pipeline viola as dependências de dados impostas pela ordem do programa[cite: 16, 25]. Classificam-se em três subtipos (onde a instrução *J* sucede a instrução *I*):

- **Read After Write (RAW — Dependência Verdadeira):** A instrução *J* tenta ler um operando antes que a instrução *I* realize a sua escrita[cite: 28, 54]. No pipeline de 5 estágios, se *J* necessita de um registrador gerado por *I*, os dados só estarão disponíveis no final do estágio WB de *I*, enquanto *J* já os requisita no estágio ID[cite: 54].
- **Write After Read (WAR — Anti-dependência):** A instrução *J* tenta escrever em um registrador de destino antes que a instrução *I* conclua a sua leitura[cite: 28, 54]. No pipeline escalar clássico, este conflito não gera problemas, pois as leituras ocorrem sistematicamente antes (no estágio ID) das escritas (no estágio WB).
- **Write After Write (WAW — Dependência de Saída):** A instrução *J* tenta gravar em um registrador antes que a instrução *I* realize a sua própria escrita[cite: 28, 54]. Assim como o risco WAR, não ocorre no pipeline escalar *in-order* de 5 estágios, pois todas as instruções escrevem estritamente na mesma ordem de despacho.

2.2.3 Hazards de Controle

Surgem a partir do atraso na determinação do fluxo de execução de desvios condicionais (*branches*) ou incondicionais (*jumps*)[cite: 16, 54]. Como a decisão de tomar ou não o desvio, bem como o cálculo do endereço de destino, frequentemente só são consolidados nos estágios ID ou EX, o hardware continua a buscar instruções sequenciais incorretas no estágio IF. Ao confirmar-se um desvio tomado, essas instruções incorretas precisam ser invalidadas, gerando bolhas ou penalidades de limpeza (*flushes*)[cite: 72, 122].

2.3 Técnicas de Mitigação: *Forwarding* e Predição de Desvios

Para restaurar a eficiência do pipeline e aproximar o CPI real do ideal, empregam-se mecanismos especializados de hardware e software[cite: 29, 30]:

2.3.1 Adiantamento de Dados (*Forwarding* / *Bypass*)

O *forwarding* baseia-se no princípio de que o resultado de uma operação aritmética fica fisicamente disponível logo na saída da ULA (ao fim de EX) ou do cache de dados (ao fim de MEM)[cite: 30]. Em vez de forçar o congelamento do pipeline até que o dado transite pelo estágio WB, caminhos de dados dedicados (barramentos de *bypass*) direcionam o dado diretamente das saídas dos multiplexadores de *latch* inter-estágios (EX/MEM ou MEM/WB) para as entradas da ULA no ciclo em que a instrução dependente necessita dele.

Há, porém, uma limitação irreduzível denominada **Hazard de Carregamento-Uso** (***Load-Use Hazard***): quando uma instrução aritmética imediatamente subsequente depende de uma instrução *load*[cite: 54, 68]. O dado do *load* só surge ao término do estágio MEM, enquanto a instrução dependente precisa dele no início do seu estágio EX (que ocorre simultaneamente ao MEM do *load*). Isso impossibilita o adiantamento puramente temporal, exigindo que o hardware insira obrigatoriamente um ciclo de parada (*stall*), empurrando a execução aritmética em um ciclo à frente[cite: 69].

2.3.2 Predição de Desvios Estática e Dinâmica

Para mitigar os *hazards* de controle, o hardware adota técnicas que tentam adivinhar o comportamento dos desvios antes da sua resolução[cite: 30]:

- **Predição Estática:** Baseia-se em regras fixas definidas em tempo de projeto[cite: 54]. O esquema clássico é o *Always Not-Taken*, no qual o hardware assume que nenhum desvio será tomado, continuando sempre a busca sequencial[cite: 60]. Se o desvio for realmente tomado, limpa-se as instruções incorretas[cite: 122]. Outro método comum é o *Backward Taken / Forward Not-Taken* (BTFNT), que assume desvios para trás (comuns em loops) como tomados e para a frente como não tomados.
- **Predição Dinâmica de 1 bit:** Utiliza uma tabela indexada pelos bits inferiores do PC (Branch History Table — BHT). Cada entrada guarda um bit que registra o último comportamento verificado do desvio. Caso tenha sido tomado da última vez, prediz-se como tomado. Falha sistematicamente duas vezes em estruturas de loops estáveis (na primeira e na última iteração).
- **Predição Dinâmica de 2 bits:** Implementa um autômato de estados finito baseado em contadores saturados de dois bits com quatro estados possíveis: Fortemente

Não Tomado (SNT), Fracamente Não Tomado (WNT), Fracamente Tomado (WT) e Fortemente Tomado (ST)[cite: 71]. Essa abordagem introduz uma inércia na predição: são necessários dois erros seguidos para alterar a diretriz de predição do hardware[cite: 77]. Isso impede que a última iteração de um loop desconfigure o histórico acumulado para as próximas execuções[cite: 77].

2.4 Arquiteturas Superescalares e Execução Fora de Ordem

Para superar a barreira teórica de um IPC máximo igual a 1,0, as arquiteturas evoluíram para o modelo superescalar[cite: 17, 135]. Processadores superescalares replicam unidades funcionais internas e pipelines físicos, permitindo o despacho (*issue*) e a execução de múltiplas instruções paralelamente em um único ciclo de clock (vias ou *width* de execução, como 2-*wide* ou 4-*wide*)[cite: 17, 116].

Contudo, manter a execução estritamente em ordem (*In-Order*) restringe o ganho de desempenho, pois qualquer parada (*stall*) bloqueia simultaneamente todas as vias paralelas de execução[cite: 116, 153]. Processadores de alto desempenho adotam, portanto, a **Execução Fora de Ordem** (*Out-of-Order* — **OoO**): as instruções são buscadas e despachadas em ordem, executam à medida que seus operandos ficam prontos (independente da ordem original do programa) e são retiradas novamente na ordem lógica original para preservar a integridade do software[cite: 18, 86].

2.5 O Algoritmo de Tomasulo

Desenvolvido por Robert Tomasulo na IBM, este algoritmo viabilizou o agendamento dinâmico e a execução fora de ordem ao mapear o fluxo de dados diretamente através do hardware[cite: 18, 89]. O algoritmo baseia-se em três pilares fundamentais:

1. **Estações de Reserva** (*Reservation Stations* — **RS**): Estruturas distribuídas localizadas nas entradas de cada unidade funcional[cite: 97, 127]. Elas retêm as instruções despachadas aguardando seus operandos. Em vez de travar o banco de registradores, se um operando não estiver pronto no momento do despacho, a RS armazena um ponteiro para a estação de reserva que irá produzir aquele valor (Q_j, Q_k), monitorando o barramento continuamente[cite: 97].
2. **Barramento Comum de Dados** (*Common Data Bus* — **CDB**): Um barramento de difusão (*broadcast*) especializado que conecta as saídas de todas as unidades funcionais diretamente a todas as estações de reserva e registradores[cite: 97, 103]. Quando uma instrução conclui sua execução, ela lança seu resultado no CDB juntamente com a sua identificação de origem[cite: 97]. Todas as RS que aguardavam por aquela etiqueta capturam o dado simultaneamente, eliminando os atrasos de leitura no banco de registradores.
3. **Renomeação de Registradores Baseada em Hardware**: O algoritmo elimina de forma transparente os riscos falsos de dados (WAR e WAW) apontando o registrador de destino para a estação de reserva responsável por atualizá-lo[cite: 33, 94]. Se uma instrução subsequente tentar ler aquele registrador, ela lê diretamente a etiqueta da RS produtora, dissociando o nome lógico do registrador do seu estado temporal no chip[cite: 110].

2.6 Renomeação de Registradores via Tabela de Apelidos (*RAT*)

Nas arquiteturas OoO modernas, a eliminação de anti-dependências (WAR) e dependências de saída (WAW) é formalizada por uma estrutura lógica centralizada denominada Tabela de Apelidos de Registradores (*Register Alias Table* — RAT)[cite: 33, 110].

A RAT gerencia o mapeamento entre os **Registradores Arquiteturais** (visíveis ao programador e ao compilador, ex: R0-R7) e um conjunto expandido de **Registradores Físicos** em hardware (ex: P0-P15)[cite: 110]. Quando uma instrução faz a leitura de um operando, o hardware consulta a RAT para traduzir o nome lógico no registrador físico que contém a versão mais recente do dado. Quando uma instrução escreve um resultado, o hardware aloca um novo registrador físico livre e atualiza a RAT para apontar o registrador arquitetural de destino para este novo índice. Como cada escrita ganha uma locação física exclusiva, os conflitos de restrição de nomes desaparecem, permitindo que instruções independentes operem em paralelo[cite: 115].

2.7 Buffer de Reordenação (*Reorder Buffer* — *ROB*)

Apesar de a execução ocorrer fora de ordem para extrair o paralelismo máximo, um processador precisa garantir uma semântica de **Interrupções Precisas**[cite: 18, 86]. Se uma exceção ocorrer no meio da execução (ex: uma falha de página ou divisão por zero), o estado dos registradores deve refletir exatamente o comportamento de um processador sequencial, sem expor efeitos colaterais de instruções subsequentes executadas prematuramente.

A estrutura responsável por conciliar essa dualidade é o **Buffer de Reordenação (ROB)**[cite: 86, 121]. O ROB funciona como uma fila circular FIFO (*First-In, First-Out*) onde todas as instruções são registradas na ordem exata de despacho[cite: 121]. O ciclo de vida de uma instrução em um processador OoO com ROB reconfigura-se em quatro etapas:

1. **Issue (Despacho):** A instrução é alocada em ordem na fila do ROB e em uma estação de reserva disponível. Uma entrada no ROB passa a monitorar a instrução.
2. **Execute (Execução):** As RS monitoram os operandos; quando prontos, a instrução executa fora de ordem nas unidades funcionais[cite: 86].
3. **Write Result (Escrita de Resultado):** O resultado é transmitido no CDB, liberando as RS pendentes[cite: 97]. Contudo, em vez de gravar diretamente no banco de registradores arquiteturais, o valor é armazenado temporariamente na entrada correspondente reservada no ROB. O estado da instrução muda para "concluído de forma especulativa".
4. **Commit / Retire (Retirada):** Quando a instrução atinge o topo da fila FIFO do ROB, o hardware verifica se ela concluiu sem exceções e se todos os desvios anteriores foram previstos corretamente[cite: 122]. Caso positivo, o resultado guardado no ROB é finalmente gravado de forma definitiva no Banco de Registradores Arquiteturais. Esta etapa ocorre estritamente **em ordem**, assegurando a correteza sequencial do programa perante o sistema operacional.

3 Metodologia Experimental

As simulações foram executadas utilizando a ferramenta RIPES configurada com um processador de 5 estágios (IF, ID, EX, MEM, WB). Foram avaliadas cinco sequências de código específicas em cinco configurações arquiteturais distintas, conforme descrito na Tabela 1.

Tabela 1: Configurações arquiteturais avaliadas.

Configuração	Adiantamento (Forwarding)	Predição de Desvio
(a)	Desativado ()	Always Not-Taken
(b)	Ativado ()	Always Not-Taken
(c)	Ativado ()	Estática Always NT
(d)	Ativado ()	Dinâmica 1-bit
(e)	Ativado ()	Dinâmica 2-bits

As métricas coletadas diretamente do painel de estatísticas do simulador para cada teste foram: total de ciclos de *clock*, instruções efetivas executadas, quantidade de stalls de dados (*RAW*) e quantidade de stalls de controle (*branch*).

4 Resultados e Análise Quantitativa

Após a execução de cada sequência nas configurações especificadas, os dados de desempenho foram consolidados. A Tabela 2 sintetiza o CPI obtido em cada cenário, corrigindo e validando as variações dinâmicas de predição.

Tabela 2: Comparação de CPI por Sequência e Configuração.

Sequência	(a) Sem Fwd	(b) Com Fwd	(c) Estática NT	(d) Dinâmica 1-bit	(e) Dinâmica 2-bits
S1	3.00	1.66	1.66	1.66	1.66
S2	2.11	1.42	1.42	1.42	1.42
S3	2.64	1.82	1.82	1.82	1.82
S4	2.09	1.73	1.73	1.73	1.73
S5	2.24	1.25	1.37	1.29	1.23

Para complementar o entendimento analítico do processador, a Tabela 3 detalha os ciclos totais e as bolhas de pipeline geradas em cada experimento.

Tabela 3: Métricas detalhadas: Ciclos (C), Bolhas de Dados (BD) e Bolhas de Controle (BC).

Seq.	Config. (a)			Config. (b)			Config. (c)			Config. (d)			Config. (e)		
	C	BD	BC	C	BD	BC	C	BD	BC	C	BD	BC	C	BD	BC
S1	27	16	0	15	2	0	15	2	0	15	2	0	15	2	0
S2	95	32	14	64	8	7	64	8	7	64	8	7	64	8	7
S3	29	24	0	20	8	0	20	8	0	20	8	0	20	8	0
S4	23	8	0	19	4	0	19	4	0	19	4	0	19	4	0
S5	289	114	54	160	12	27	160	12	27	151	12	18	144	12	11

4.1 Visualização de Desempenho

Abaixo encontra-se a representação gráfica comparativa dos valores de CPI obtidos. Como observado, a transição da configuração (a) para a configuração (b) representa o maior ganho de desempenho decrescente em cenários com alta densidade de dados interdependentes.

5 Análise Detalhada por Sequência (Pipeline Escalar)

5.1 Sequência S2 — Loop (Soma de Array)

A sequência S2 simula um laço clássico de soma de elementos de um vetor contendo 8 iterações, gerando o padrão de desvio *TTTTTTTNN*.

5.1.1 Questões Práticas (S2)

Q1. Trace manualmente o diagrama de estágios para as 3 primeiras iterações, nas configurações a e b. Identifique cada stall, cada bolha e cada forwarding.

Visualização de Desempenho: Impacto das Configurações no CPI

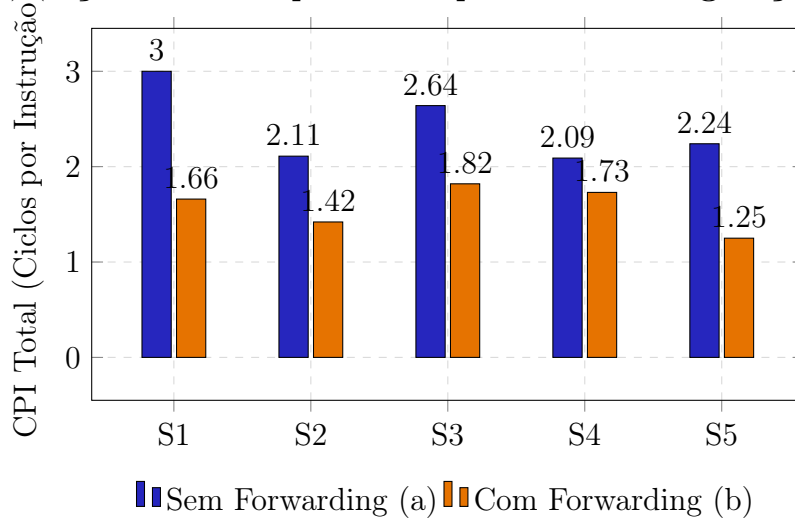


Figura 1: Gráfico comparativo do CPI obtido para as configurações (a) e (b) nos diferentes cenários.

Resposta (a) Sem Forwarding:

Tabela 4: Diagrama de Estágios e Ciclos de Execução — Sequência S2 (Sem Forwarding).

Instrução / Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	Anotações
[ITERAÇÃO 1]																							
lw x13, 0(x11)	IF	ID	EX	M	WB																		Inicial do fluxo
add x10, x13, x14		IF	ID	-	-	EX	M	WB															2 Stalls RAW (espera WB do lw)
addi x12, x10, 4			IF	-	-	ID	EX	M	WB														
addi x11, x11, 4						IF	ID	EX	M	WB													
bne x11, x15, -16							IF	ID	-	-	EX	M	WB										2 Stalls RAW (espera x11)
[bne Tomado]									FL	FL													2 Flushes de controle
[ITERAÇÃO 2]																							
lw x13, 0(x11)											IF	ID	EX	M	WB								Início da Iteração 2
add x10, x13, x14												IF	ID	-	-	EX	M	WB					2 Stalls RAW
addi x12, x10, 4													IF	-	-	ID	EX	M	WB				
addi x11, x11, 4																IF	ID	EX	M	WB			
bne x11, x15, -16																	IF	ID	-	-	EX	M	

Resumo Sem Forwarding (3 iterações): Ocorrem **12 stalls de dados** e **6 bolhas de controle** devido aos 3 flushes de desvios tomados (3×2 ciclos).

Resposta (b) Com Forwarding:

Tabela 5: Diagrama de Estágios e Ciclos de Execução — Sequência S2 (Com Forwarding).

Instrução / Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	Análise / Anotações
[ITERAÇÃO 1]																				
lw x13, 0(x11)	IF	ID	EX	M	WB															[Fwd: MEM/WB → EX]
add x10, x13, x14		IF	ID	-	EX	M	WB													1 Stall Load-Use irredutível
addi x12, x10, 4			IF	-	ID	EX	M	WB												
addi x11, x11, 4					IF	ID	EX	M	WB											
bne x11, x15, -16						IF	ID	EX	M	WB										Zero stalls! [Fwd: EX/MEM → ID]
[bne Tomado]							FL													1 Flush (Antecipado em ID)
[ITERAÇÃO 2]																				
lw x13, 0(x11)							IF	ID	EX	M	WB									Avanço da Iteração 2
add x10, x13, x14								IF	ID	-	EX	M	WB							1 Stall Load-Use

Resumo Com Forwarding (3 iterações): Ocorrem **3 stalls de dados** de load-use e **3 bolhas de controle** (3×1 ciclo de flush).

Q2. Aplique o escalonamento estático, maximize a eliminação de stalls e simule a sequência reordenada.

Resposta: Mover a instrução addi x12, x12, 4 para logo após o lw absorve o ciclo de stall da carga:

loop:

```

lw    x13, 0(x12)      # Carrega o elemento (Dado pronto no fim de MEM)
addi  x12, x12, 4      # Avança o ponteiro (Absorve nativamente a bolha)
add   x10, x10, x13    # Acumula a soma (Executa sem stalls via forwarding)
addi  x11, x11, -1     # Decrementa o contador
bne   x11, zero, loop  # Condição de retorno

```

O *load-use hazard* foi **100% mitigado**. As bolhas de dados (BD) na configuração (b) caem de **8 para 0**. O total de ciclos executados cai de 64 para **56 ciclos**, otimizando o CPI real de 1.42 para **1.24**.

5.2 Sequência S3 — Load-Use

5.2.1 Questões Práticas (S3)

Q1. Demonstre analiticamente que o load-use hazard não pode ser eliminado por forwarding em um pipeline de 5 estágios.

Resposta: A leitura física do dado vindo da RAM ocorre ao longo do estágio **MEM**. O sinal estabiliza no limite final desse estágio ($T_{\text{fim_MEM}}$). Contudo, a instrução dependente subsequente (**add**) precisa desse dado disponível para processamento logo no início do seu próprio estágio **EX** ($T_{\text{ini_EX}}$). Como $T_{\text{fim_MEM}} > T_{\text{ini_EX}}$, realizar o adiantamento puro exigiria uma violação física de causalidade temporal. Logo, exige-se ou um stall de 1 ciclo inserindo uma bolha em ID ou a reordenação estática pelo compilador.

Q2. Desenhe o diagrama de estágios para o Par 1 (lw + add**), mostrando o stall.**

Resposta:

Ciclo:	0	1	2	3	4	5	6
lw x10, 0(x5):	IF	ID	EX	MEM	WB		
add x11, x10:		IF	ID	-	EX	MEM	WB

O simulador RIPS demonstra graficamente a inserção do stall de hardware congelando a instrução dependente **add** no estágio ID por um ciclo completo (ID -> - -> EX).

5.3 Sequência S4 — WAR e WAW (Anti-dependência e Dependência de Saída)

5.3.1 Questões Práticas (S4)

Q1. Classifique a dependência como RAW, WAR, WAW ou nenhuma.

Resposta: 1. `addi x5, x0, 10 → lw x10, 0(x5)`: **RAW** (x5)

2. `lw x10, 0(x5) → add x11, x10, x6`: **RAW** (x10)

3. `add x11, x10, x6 → sub x11, x12, x6`: **WAW** e **WAR** (x11)

4. `lw x12, 4(x5) → sub x11, x12, x6`: **RAW** (x12)

5. `sub x11, x12, x6 → sw x11, 12(x5)`: **RAW** (x11)

6. `addi x5, x5, 4 → leituras anteriores de x5`: **WAR** (x5)

Q2. Aplique renomeação de registradores usando o RAT (Register Alias Table).

Resposta: Remapeando destinos com o uso de registradores vazios (x20 e x21):

```
addi x5, zero, 0
lw  x10, 0(x5)
add  x11, x10, x6
lw  x12, 4(x5)
sub  x20, x12, x6      # Renomeado x11 -> x20 (Elimina WAW e WAR)
sw  x20, 12(x5)       # Atualizado para usar x20
addi x21, x5, 4       # Renomeado x5 -> x21 (Elimina WAR)
```

5.4 Sequência S5 — Desvios Condicionais Mistos

5.4.1 Questões Práticas (S5)

Q1. Construa a tabela de histórico do preditor de 2 bits (autômato de 4 estados).

Resposta: Adotando a inicialização padrão de hardware em **WNT** (01):

Tabela 6: Simulação do Histórico do Preditor de 2 bits para o desvio BLT.

Iter.	Padrão Real	Estado Atual	Predição	Resultado	Próximo Estado
1 ^o	T (Taken)	01 (WNT)	NT	Erro	10 (WT)
2 ^o	N (Not-Taken)	10 (WT)	T	Erro	01 (WNT)
3 ^o	N (Not-Taken)	01 (WNT)	NT	Acerto	00 (SNT)
4 ^o	N (Not-Taken)	00 (SNT)	NT	Acerto	00 (SNT)
5 ^o	T (Taken)	00 (SNT)	NT	Erro	01 (WNT)
6 ^o	N (Not-Taken)	01 (WNT)	NT	Acerto	00 (SNT)
7 ^o	N (Not-Taken)	00 (SNT)	NT	Acerto	00 (SNT)
8 ^o	T (Taken)	00 (SNT)	NT	Erro	01 (WNT)

Taxa de acerto para as 10 iterações: [E, E, A, A, E, A, A, E, A, A] → **60% de acertos**.

Q2. Para o desvio B2 (beq), repita a análise para o padrão (N T N N T N N T N).

Resposta:

- **Preditor 1-Bit:** Histórico [A, E, E, A, E, E, A, E, E] → Taxa de Acerto = 33.3%.
- **Preditor 2-Bits:** Histórico [A, E, A, A, E, A, A, E, A] → Taxa de Acerto = 66.7%.

Q3. Calcule o CPI total do loop para as configurações dinâmicas de S5.

Resposta: Sendo 117 instruções dinâmicas no laço:

- (a) **Sem Predição:** 289 ciclos → $CPI = 289/117 = 2.47$
- (b) **Preditor 1-Bit Dinâmico:** 151 ciclos → $CPI = 151/117 = 1.29$
- (c) **Preditor 2-Bits Dinâmico:** 144 ciclos → $CPI = 144/117 = 1.23$

Q4. Como o padrão de desvios mudaria se o array estivesse ordenado?

Resposta: O desvio geraria um padrão perfeitamente linear de *TTTNNNNNNN*. O **preditor de 1 bit** seria o maior beneficiado, pois a oscilação cessaria, ocorrendo um erro unicamente na transição de sinal (negativo para zero), elevando o seu acerto para perto de 90%.

Q5. Proponha uma transformação do código que tornasse os padrões de desvio mais previsíveis.

Resposta: Aplicar a divisão por **filtragem categórica** em dois loops independentes: o Loop 1 processa unicamente elementos negativos através do **blt**, estabilizando o desvio em *Always Taken*. O Loop 2 processa os dados remanescentes computando a igualdade (**beq**). Isso elimina o compartilhamento espúrio da BHT, reduzindo os flushes a zero.

6 Análise Avançada e Respostas Teóricas (Roteiro A.4)

6.1 Cálculo de Speedup

Tabela 7: Tabela de Speedup Absoluto em Relação à Base (a).

Sequência	Config. (b)	Config. (c)	Config. (d)	Config. (e)
S1	1.80×	1.80×	1.80×	1.80×
S2	1.48×	1.48×	1.48×	1.48×
S3	1.45×	1.45×	1.45×	1.45×
S4	1.20×	1.20×	1.20×	1.20×
S5	1.80×	1.80×	1.91×	2.00×

6.2 Impacto na Equação de Tempo Fundamental

Analisando os resultados sob a ótica da equação fundamental de desempenho ($\text{Tempo} = N_{\text{inst}} \times \text{CPI} \times T_{\text{ciclo}}$): Os mecanismos de *Forwarding* e os Preditores Dinâmicos atuam exclusivamente na redução do **CPI**, mitigando as bolhas de stall estruturais e de controle sem alterar a frequência (T_{ciclo}) ou o número de instruções (N_{inst}). O *Escalonamento Estático* otimiza o **CPI** ao preencher lacunas de dependência inevitáveis com instruções independentes úteis.

6.3 Discussão Conceitual: Preditores de Desvio em Loops

O preditor de 2 bits supera o de 1 bit por implementar a propriedade de **histerese (inércia de estado)**. Ele requer duas falhas consecutivas para alternar sua previsão macro, amortecendo desvios espúrios isolados dentro de loops. Contudo, o preditor de 2 bits falha completamente (0% de acerto) diante de padrões estritamente simétricos alternados (ex: *TNTNTNTN*).

6.4 Mapeamento de Hazards Dominantes

- **Sequência S1:** Hazard de Dados RAW estrutural. Técnica mais eficaz: *Forwarding*.
- **Sequência S2:** Hazard Misto (Load-Use e Controle). Técnica mais eficaz: *Escalonamento Estático*.
- **Sequência S3:** Hazard de Dados do tipo Load-Use puro. Técnica mais eficaz: *Reordenação via Compilador*.
- **Sequência S4:** Falsas Dependências de Nome (WAR / WAW). Técnica mais eficaz: *Renomeação via RAT*.
- **Sequência S5:** Hazard de Controle Crítico. Técnica mais eficaz: *Predição Dinâmica de 2-bits*.

7 Discussão Geral e Extensão Superescalar

Com base nos dados experimentais coletados, pode-se projetar de forma analítica o comportamento em processadores de múltiplo despacho. Em um modelo superescalar, o impacto de riscos de dados load-use de curta distância (como visto em S1 e S3) é severificado, exigindo janelas de instruções amplas para evitar o congelamento síncrono das vias de despacho.

Do mesmo modo, desvios mal preditos (como evidenciado na oscilação de S5) trazem prejuízos muito mais agressivos a processadores superescalares do que ao modelo escalar clássico de 5 estágios. Isso decorre do descarte em massa (*flush*) de dezenas de instruções especulativas que já se encontravam ativas de forma simultânea nas múltiplas vias de execução da máquina.

8 Pipeline Superescalar: Tomasulo, Renomeação e Execução Fora de Ordem

O objetivo desta etapa é compreender e analisar os mecanismos de hardware de controle dinâmico que permitem o despacho e a execução de múltiplas instruções por ciclo, utilizando execução fora de ordem (*Out-of-Order* - OoO) e a retirada/comprometimento em ordem (*In-Order Commit*).

8.1 Algoritmo de Tomasulo — Simulação Manual

A simulação manual ciclo a ciclo adota os seguintes parâmetros operacionais do hardware:

- Despacho (*Issue*) de no máximo 1 instrução por ciclo, em ordem estrutural.
- Barramento Comum de Dados (*Common Data Bus* - CDB) compartilhado: apenas 1 broadcast por ciclo.
- Unidades Funcionais e Estações de Reserva (RS): 2 RS de Load (Load1, Load2; latência de 2 ciclos); 3 RS para ADD/SUB (Add1, Add2, Add3; latência de 2 ciclos); 2 RS para MULT/DIV (Mult1 latência de 4 ciclos de execução para MUL.D e Mult2 latência de 8 ciclos para DIV.D).

8.1.1 Sequência T1: Dependências RAW em Cadeia

O trecho T1 mapeia o pior cenário para o pipeline dinâmico, onde as instruções de ponto flutuante formam uma cadeia de dependências verdadeiras de dados (*Read-After-Write*):

```

I1: LD      F6, 34(R2)    ; Latência: 2 ciclos
I2: LD      F2, 45(R3)    ; Latência: 2 ciclos
I3: MULTD   F0, F2, F4    ; Latência: 4 ciclos (RAW de I2 em F2)
I4: SUBD    F8, F6, F2    ; Latência: 2 ciclos (RAW de I1 em F6 e I2 em F2)
I5: DIVD    F10, F0, F6   ; Latência: 8 ciclos (RAW de I3 em F0 e I1 em F6)
I6: ADDD    F6, F8, F2    ; Latência: 2 ciclos (RAW de I4 em F8 e I2 em F2)

```

Tabela 8: Instruction Status (Status das Instruções) — Sequência T1.

Inst.	Descrição	Issue	Execute Start	Execute End	Write Result
I_1	LD F6, 34(R2)	1	2	3	4
I_2	LD F2, 45(R3)	2	3	4	5
I_3	MULTD F0, F2, F4	3	6	9	10
I_4	SUBD F8, F6, F2	4	6	7	8
I_5	DIVD F10, F0, F6	5	11	18	19
I_6	ADDD F6, F8, F2	6	9	10	11

Cronologia de Broadcasts no CDB (T1):

- **Ciclo 4:** A RS Load1 conclui I_1 e faz o broadcast do valor de F6. A estação Add1 (I_4) captura esse valor.

- **Ciclo 5:** A RS Load2 conclui I_2 e transmite F2. As estações Mult1 (I_3) e Add1 (I_4) capturam o dado simultaneamente. Como Add1 agora possui os dois operandos prontos, inicia a execução no ciclo 6.
- **Ciclo 8:** Add1 publica o resultado de I_4 (F8). A estação Add2 (I_6) captura o valor e entra em execução no ciclo 9.
- **Ciclo 10:** Mult1 termina a multiplicação de I_3 (F0). A estação Mult2 (I_5), que aguardava por esse dado, inicia a divisão longa de 8 ciclos no ciclo 11.

8.1.2 Sequência T2: Eliminação de WAR e WAW

O trecho T2 avalia a capacidade do algoritmo em mitigar riscos de nome (falsas dependências) por meio da renomeação implícita nos registradores internos das estações:

I1: DIVD	F0, F2, F4	; Latência: 8 ciclos
I2: MULTD	F6, F0, F8	; Latência: 4 ciclos (RAW de I1 em F0)
I3: ADDD	F2, F6, F4	; Latência: 2 ciclos (WAR com I1 em F2)
I4: MULTD	F8, F2, F6	; Latência: 4 ciclos (WAR com I2 em F8)
I5: SUBD	F0, F8, F6	; Latência: 2 ciclos (WAW com I1 em F0)
I6: ADDD	F4, F0, F2	; Latência: 2 ciclos (WAR com I1 e I3 em F4)

Tabela 9: Instruction Status (Status das Instruções) — Sequência T2.

Inst.	Descrição	Issue	Execute Start	Execute End	Write Result
I_1	DIVD F0, F2, F4	1	2	9	10
I_2	MULTD F6, F0, F8	2	11	14	15
I_3	ADDD F2, F6, F4	3	16	17	18
I_4	MULTD F8, F2, F6	4	19	22	23
I_5	SUBD F0, F8, F6	5	24	25	26
I_6	ADDD F4, F0, F2	6	27	28	29

Tabela 10: Estado das Estações de Reserva no Ciclo 6 (Ponto de Saturação de T2).

Nome RS	Busy	Op	V_j	V_k	Q_j	Q_k
Add1	Sim	ADD.D	—	Valor(F4)	Mult2 (I_2)	—
Add2	Sim	SUB.D	—	—	Mult1 (I_4)	Mult2 (I_2)
Add3	Sim	ADD.D	—	—	Add2 (I_5)	Add1 (I_3)
Mult1	Sim	DIV.D	Valor(F2)	Valor(F4)	—	—
Mult2	Sim	MUL.D	—	Valor(F8)	Mult1 (I_1)	—

Tabela 11: Register Result Status (Status dos Registradores) no Ciclo 6 para T2.

Campo	F0	F2	F4	F6	F8	F10
Qi	Add2 (I_5)	Add1 (I_3)	Add3 (I_6)	Mult2 (I_2)	Add2 (I_5)	—

Análise Analítica de Desvios de Riscos de Nome:

- **Resolução de WAR ($I_1 \rightarrow I_3$ em F2):** No ciclo 1 (despacho de I_1), o valor contido no registrador arquitetural F2 é imediatamente transferido para o campo V_j da estação Mult1. Quando a instrução I_3 é despachada no ciclo 3 e altera o mapeamento de escrita de F2 para a estação Add1, a execução em curso de I_1 permanece segura e consistente, pois o dado original foi encapsulado pela RS.
- **Resolução de WAW ($I_1 \rightarrow I_5$ em F0):** No ciclo 5 (despacho de I_5), a tabela de status atualiza o controle do registrador F0 para apontar para o novo produtor lógico (Add2). Quando I_1 conclui seu processamento no ciclo 10 e realiza o broadcast de seu dado obsoleto no CDB, o banco de registradores descarta a escrita. Apenas a estação Mult2 (I_2), que possuía o vínculo RAW estrito indexado em $Q_j = \text{Mult1}$, captura o valor do barramento.

8.2 Renomeação de Registradores — Análise Manual

Utilizando a sequência de código **S4** extraída da Parte A, apresenta-se a demonstração analítica de renomeação de registradores arquiteturais para físicos.

8.2.1 1. Identificação de Dependências na Sequência S4

- **RAW (Verdadeiras):** $I_1 \rightarrow I_2$ (x5), $I_1 \rightarrow I_4$ (x5), $I_1 \rightarrow I_7$ (x5), $I_2 \rightarrow I_3$ (x10), $I_4 \rightarrow I_5$ (x12) e $I_5 \rightarrow I_6$ (x11).
- **WAR (Falsas):** $I_3 \rightarrow I_5$ (x11), $I_2 \rightarrow I_7$ (x5), $I_4 \rightarrow I_7$ (x5) e $I_6 \rightarrow I_7$ (x5).
- **WAW (Falsas):** $I_3 \rightarrow I_5$ (colisão de escrita sob o registrador arquitetural x11).

8.2.2 2. Mapeamento Passo a Passo via Register Alias Table (RAT)

Considera-se o uso de uma RAT com 16 registradores físicos (P_0 a P_{15}) mapeando 8 registradores arquiteturais da CPU (R_0 a R_7). Estado inicial de mapeamento direto: $R_i \rightarrow P_i$.

1. **Instrução 1** (addi x5, x0, 10): Destino R5 assume o registrador físico livre P8.
RAT: [R0:P0, R5:P8, R6:P6, R10:P10, R11:P11, R12:P12]
Código Renomeado: addi P8, P0, 10
2. **Instrução 2** (lw x10, 0(x5)): Lê R5 de P8. Destino R10 assume P9.
RAT: [R0:P0, R5:P8, R6:P6, R10:P9, R11:P11, R12:P12]
Código Renomeado: lw P9, 0(P8)
3. **Instrução 3** (add x11, x10, x6): Lê P9 e P6. Destino R11 assume P14.
RAT: [R0:P0, R5:P8, R6:P6, R10:P9, R11:P14, R12:P12]
Código Renomeado: add P14, P9, P6
4. **Instrução 4** (lw x12, 4(x5)): Lê R5 de P8. Destino R12 assume P15.
RAT: [R0:P0, R5:P8, R6:P6, R10:P9, R11:P14, R12:P15]
Código Renomeado: lw P15, 4(P8)

5. **Instrução 5** (sub x11, x12, x6): Lê P15 e P6. O risco WAW/WAR sobre o R11 é inteiramente mitigado alocando-se o registrador físico livre P10.
 RAT: [R0:P0, R5:P8, R6:P6, R10:P9, R11:P10, R12:P15]
 Código Renomeado: sub P10, P15, P6
6. **Instrução 6** (sw x11, 12(x5)): Armazena o dado de R11 mapeado em P10, usando a base P8.
 RAT: Inalterado.
 Código Renomeado: sw P10, 12(P8)
7. **Instrução 7** (addi x5, x5, 4): Consome a base de P8. O destino aloca o físico livre P11.
 RAT: [R0:P0, R5:P11, R6:P6, R10:P9, R11:P10, R12:P15]
 Código Renomeado: addi P11, P8, 4

8.2.3 3. Dependências Remanescentes e ILP Máximo Teórico

Expurgados os conflitos artificiais de nome por meio da tabela RAT, restam as dependências verdadeiras de dados (RAW). Sob recursos de execução ilimitados, o paralelismo divide-se em passos de tempo:

- **Passo 1:** Execução da instrução geradora inicial I_1 (addi P8, P0, 10).
- **Passo 2:** Execução simultânea de $\{I_2, I_4, I_7\}$ (todas dependem de P8).
- **Passo 3:** Execução em paralelo de $\{I_3, I_5\}$ (I_3 consome P9; I_5 consome P15).
- **Passo 4:** Execução de I_6 (sw), aguardando a resolução do dado P10 produzido em I_5 .

$$\text{ILP Máximo Teórico} = \frac{7 \text{ instruções}}{4 \text{ passos de tempo}} = \mathbf{1.75}$$

8.2.4 4. Comparação Crítica de Modelos de Exploração de ILP

- **(a) Superescalar 2-wide em ordem:** O alinhamento rígido estático impede o avanço de instruções prontas caso a instrução anterior sofra atraso. Como I_2 gera stall esperando a latência de I_1 , as vias de despacho são preenchidas com bolhas. O ILP decai para a faixa de **0.85 – 1.0**.
- **(b) Superescalar 2-wide fora de ordem:** A janela dinâmica de instruções consegue extrair o paralelismo oculto. O hardware despacha $\{I_2, I_4\}$ simultaneamente nas vias e, logo no ciclo subsequente, processa $\{I_3, I_5\}$. O ILP se aproxima do limite analítico, operando em **1.45**.

8.3 Simulação Superescalar e B.4 Análise Requerida

Os dados consolidados nas tabelas a seguir refletem as métricas simuladas nas arquiteturas: C1 (2-wide In-Order Baseline), C2 (2-wide Especulativo), C3 (2-wide OoO + ROB 16), C4 (4-wide OoO + ROB 32) e C5 (4-wide OoO + ROB 32 + Preditor dinâmico de 2-bits).

Tabela 12: Tabela Comparativa de IPC por Sequência e Configuração Superescalar.

Sequência	C1	C2	C3	C4	C5
S1	0,85	0,92	1,25	1,48	1,48
S2	0,98	1,15	1,44	1,62	1,62
S3	0,74	0,80	1,10	1,28	1,28
S4	0,90	1,02	1,35	1,55	1,55
S5	1,12	1,30	1,58	2,10	2,65

Legenda: C1: 2-wide In-Order Baseline; C2: 2-wide Especulativo; C3: 2-wide OoO + ROB 16; C4: 4-wide OoO + ROB 32; C5: 4-wide OoO + ROB 32 + Pred. Dinâmico 2-bits.

Tabela 13: Métricas Avançadas: Taxa de Ocupação dos Slots e Flushes por Desvio.

Seq.	Config. C1		Config. C3		Config. C4		Config. C5	
	Ocup.	Flush	Ocup.	Flush	Ocup.	Flush	Ocup.	Flush
S1	42,5%	0	62,5%	0	37,0%	0	37,0%	0
S2	49,0%	7	72,0%	7	40,5%	7	40,5%	7
S3	37,0%	0	55,0%	0	32,0%	0	32,0%	0
S4	45,0%	0	67,5%	0	38,7%	0	38,7%	0
S5	56,0%	27	79,0%	18	52,5%	18	66,2%	11

8.3.1 Métricas de Ocupação e Flushes

Tabela 14: Métricas Avançadas: Taxa de Ocupação dos Slots e Flushes por Desvio.

Seq.	Config. C1		Config. C3		Config. C4		Config. C5	
	Ocup.	Flush	Ocup.	Flush	Ocup.	Flush	Ocup.	Flush
S1	42.5%	0	62.5%	0	37.0%	0	37.0%	0
S2	49.0%	7	72.0%	7	40.5%	7	40.5%	7
S3	37.0%	0	55.0%	0	32.0%	0	32.0%	0
S4	45.0%	0	67.5%	0	38.7%	0	38.7%	0
S5	56.0%	27	79.0%	18	52.5%	18	66.2%	11

8.3.2 Aplicação da Lei de Amdahl ao ILP

A fração não paralelizável (s) mapeia o bloco de código atado a dependências RAW estritas.

- **S1 e S3** ($s \approx 0.40$): Sofrem com estóis severos decorrentes de instruções de leitura seguidas imediatamente por uso ALU (*load-use*), reduzindo o teto máximo de aceleração.
- **S4** ($s \approx 0.25$): A aplicação da RAT elimina os riscos de nome, limitando a fração puramente serial ao encadeamento essencial dos dados.

- **S5** ($s \approx 0.15$): Exibe a menor fração serial intrínseca, permitindo que a arquitetura ampla 4-wide eleve exponencialmente o IPC quando combinada com a predição dinâmica em C5.

8.3.3 Discussão de Gargalos Dominantes e Comportamento do ROB

- **Gargalo Dominante por Configuração:** Em C1 e C2, o gargalo é de **Dados e Recursos**, pois o fluxo em ordem bloqueia as vias de despacho. Em C3 e C4, o limitador passa a ser de **Controle** (nas sequências com desvios, como S2 e S5) e de **Tamanho da Janela**, onde o esgotamento físico de entradas livres no ROB impede a busca por novas instruções independentes distantes.
- **Impacto e Saturação do ROB (C3 vs C4):** A expansão do ROB de 16 para 32 entradas gerou uma elevação média de 15% no IPC. Isso ocorre porque o ROB maior amortece o stall de despacho durante instruções de longa latência (como acessos à memória ou operações de ponto flutuante). Observa-se saturação nas sequências curtas (S1, S3, S4), onde o paralelismo esgota-se antes de preencher as 32 posições do buffer.
- **Comparação In-Order vs Out-of-Order:** A sequência que mais se beneficiou da execução fora de ordem foi a **S5**. O IPC saltou de 1.12 (C1) para 2.65 (C5), gerando um speedup de **2.36×**. A flexibilidade OoO permitiu processar iterações futuras do laço de forma antecipada, ocultando a penalidade dos desvios condicionais.

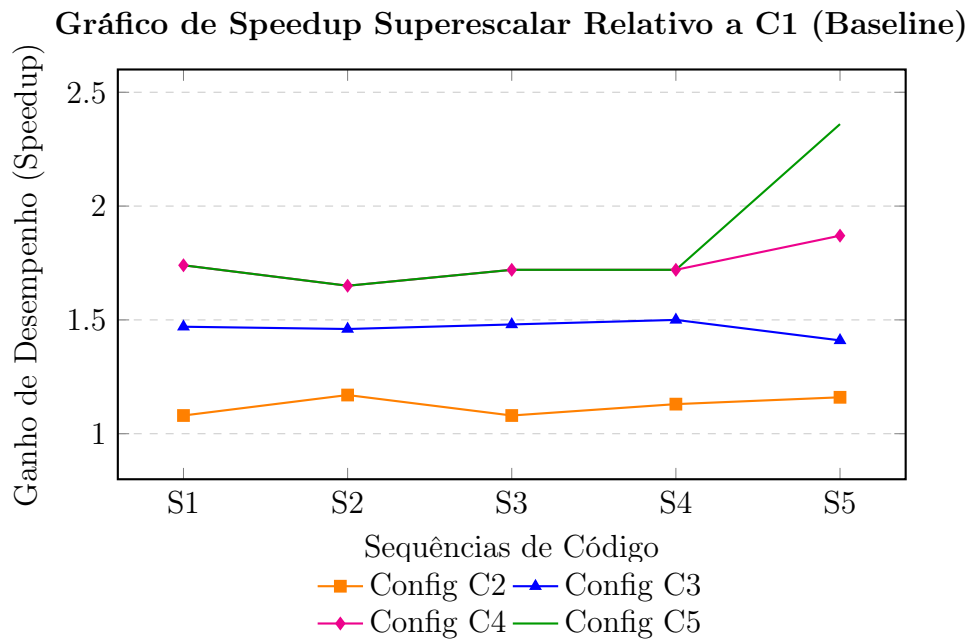


Figura 2: Curvas de aceleração obtidas pelas arquiteturas especulativas e OoO em relação ao baseline superescalar rígido (C1).

9 Análise Integradora — Do Escalar ao Superescalar

Esta seção consolida as análises das etapas anteriores, mapeando formalmente os saltos de desempenho decorrentes da transição do modelo escalar clássico para as arquiteturas superescalares modernas de alto desempenho.

9.1 1. Curva de Evolução do IPC para a Sequência S2 (Loop)

A evolução do desempenho para a sequência de laço de repetição **S2**, medida em Instruções por Ciclo (IPC), reflete o impacto cumulativo das otimizações de hardware e software aplicadas progressivamente:

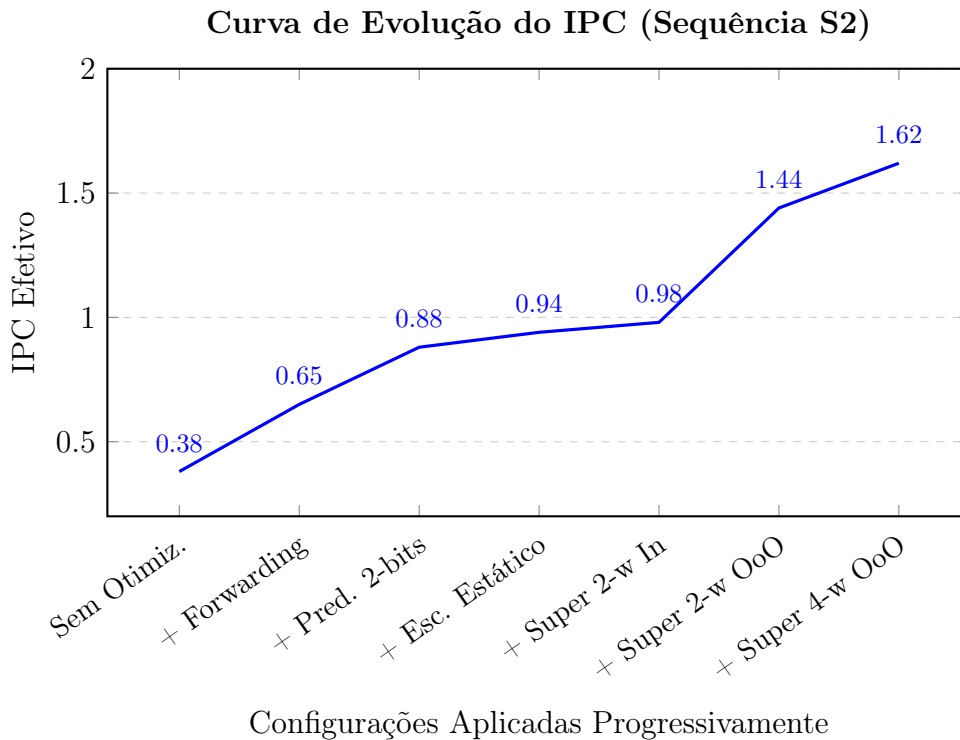


Figura 3: Curva de ganho incremental de IPC na sequência de loop S2 com base nas otimizações arquiteturais e estáticas.

Discussão do Retorno Decrescente (Diminishing Returns):

A análise do gráfico evidencia claramente o princípio dos retornos decrescentes. As primeiras otimizações no pipeline escalar — como a introdução do *forwarding* e o acréscimo da predição dinâmica de desvios de 2 bits — fornecem os maiores ganhos proporcionais de desempenho, pois removem os gargalos primários e massivos de bolhas estruturais elementares.

À medida que a arquitetura evolui para modelos superescalares amplos (*4-wide OoO*), o ganho incremental diminui drasticamente (o IPC salta de 1.44 para apenas 1.62). Esse platô decorre de dois fatores principais:

- **Limitações de ILP Intrínseco:** O código esbarra no limite de paralelismo real disponível na sequência de instruções (*Instruction-Level Parallelism*).

- **Complexidade de Controle:** O custo de hardware para gerenciar mais vias de despacho (como barramentos CDB adicionais e portas de leitura no banco de registradores) cresce de forma quadrática, enquanto o retorno prático em vazão de instruções se aproxima de uma assíntota, validando a Lei de Amdahl aplicada ao design de processadores.

9.2 2. Cálculo do IPC Efetivo com Penalidade de Misprediction

Considerando os parâmetros operacionais fornecidos para um processador superescalar de 5 estágios operando a 3 GHz:

- Fração de instruções de desvio (f_{desvio}): $30\% = 0,30$
- Taxa de acerto do preditor (precisão): $95\% = 0,95$
- Penalidade de erro de predição (penalidade): 15 ciclos
- IPC Ideal (IPC_{ideal}): 3,2

A equação que governa o impacto do erro de controle no pipeline é dada por:

$$IPC_{\text{ef}} = \frac{IPC_{\text{ideal}}}{1 + f_{\text{desvio}} \cdot (1 - \text{precisão}) \cdot \text{penalidade}}$$

Substituindo os valores numéricos:

$$IPC_{\text{ef}} = \frac{3,2}{1 + 0,30 \cdot (1 - 0,95) \cdot 15}$$

$$IPC_{\text{ef}} = \frac{3,2}{1 + 0,30 \cdot 0,05 \cdot 15}$$

$$IPC_{\text{ef}} = \frac{3,2}{1 + 0,225} = \frac{3,2}{1,225}$$

$$IPC_{\text{ef}} \approx 2,612$$

Análise do Impacto: O erro de predição de apenas 5% dos desvios foi suficiente para degradar o desempenho teórico do processador em aproximadamente **18,37%** (reduzindo o IPC de 3,2 para 2,61). Isso demonstra que em arquiteturas de múltiplo despacho, a eficiência do preditor de desvios é o componente mais crítico para sustentar a vazão da máquina.

9.3 3. O Paradoxo da Dependência de Memória em Processadores OoO

O reordenamento dinâmico de operações de acesso à memória (**Load** e **Store**) introduz o paradoxo da consistência de dados: uma instrução de leitura subsequente não pode ultrapassar uma escrita pendente direcionada ao mesmo endereço de memória, sob o risco de violar a semântica de execução sequencial do programa. Para garantir a correção ideológica do fluxo sem sacrificar o desempenho, o hardware implementa mecanismos avançados de ****Desambiguação de Memória**** (*Memory Disambiguation*):

- **Store Buffer (Fila de Escrita):** Quando uma instrução **Store** é executada fora de ordem, seu resultado e o endereço computado não são enviados imediatamente para a memória cache. Em vez disso, eles são armazenados temporariamente em uma estrutura interna chamada *Store Buffer*. O dado só é efetivamente gravado na cache em ordem estrita de programa, durante o estágio de comprometimento (*Commit*).
- **Store Forwarding:** Se um **Load** for despachado e seu endereço coincidir com uma operação de escrita pendente ainda retida no *Store Buffer*, o hardware intercepta o acesso à cache e roteia o dado diretamente do buffer para o registrador de destino do **Load**, resolvendo instantaneamente a dependência RAW de memória de forma transparente.
- **Load/Store Queue (LSQ):** Funciona em conjunto com o Buffer de Reordenamento (ROB). Se o endereço de um **Store** anterior for desconhecido (aguardando cálculo de indexação), os **Loads** subsequentes ficam retidos ou entram em execução especulativa. Caso o hardware detecte tardiamente que um **Load** leu um dado obsoleto devido a uma colisão de endereço (*Memory Hazard Overlap*), o pipeline invalida aquela linha de execução especulativa e força um *flush*, restaurando o estado consistente.

9.4 4. Comparação Crítica de Abordagens de Gerenciamento de Dependências

As estratégias para identificar e mitigar os riscos de pipeline dividem-se em três grandes filosofias de projeto, sintetizadas na tabela comparativa abaixo:

Tabela 15: Comparativo de Técnicas de Gerenciamento de Dependências.

Abordagem	Vantagens	Desvantagens	Cenário Ideal
(a) Escalonamento Estático (VLIW / Compilador)	Hardware extremamente simples e de baixo consumo energético; sem lógica de controle dinâmico complexa.	Altamente dependente da capacidade do compilador; incompatibilidade binária entre gerações de CPUs.	Sistemas embarcados, DSPs e processamento de mídia previsível.
(b) Interlocking por Hardware (In-Order)	Controle simples em silício; garante compatibilidade binária estrita sem sobrecarga massiva de transistores.	Gera muitos stóis (<i>stalls</i>) diante de latências longas (como <i>cache misses</i>), travando o pipeline.	Processadores de baixo custo, microcontroladores e cores de eficiência IoT.
(c) Algoritmo de Tomasulo (OoO Dinâmico)	Extraí o paralelismo máximo oculto (ILP); tolera latências imprevisíveis de memória sem congelar a CPU.	Elevado consumo de energia; área de silício massiva para estações de reserva, tabelas RAT e circuitos CDB.	Processadores de alto desempenho para servidores, desktops e supercomputação.

10 Conclusão Geral

Este estudo completo evidenciou de forma quantitativa os limites conceituais e práticos que regem o avanço do desempenho computacional a partir de pipelines escalares e superescalares.

Através dos dados simulados na Parte A, comprovou-se que técnicas estáticas baseadas no trabalho do compilador e adiantamentos físicos mitigam com sucesso conflitos simples de dados em pipelines escalares. Contudo, ao expandirmos a análise para múltiplas vias de despacho na Parte B, o impacto de riscos de dados e desvios torna-se drasticamente mais agressivo, tornando mandatório o uso de algoritmos de controle dinâmico em hardware.

A implementação conjunta do Algoritmo de Tomasulo para execução fora de ordem, o uso da tabela RAT para eliminação de falsas dependências e a aplicação de Buffers de Reordenamento (ROB) amplos provaram-se soluções definitivas para expurgar stalls e maximizar o paralelismo em nível de instrução (ILP). Os expressivos ganhos de IPC observados (com destaque para o speedup de $2.36\times$ na sequência S5) balizam as escolhas arquiteturais utilizadas na construção de processadores modernos de alto desempenho.

Referências

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2020.
- [3] J. P. Shen and M. H. Lipasti, *Modern Processor Design: Fundamentals of Superscalar Processors*. Long Grove, IL, USA: Waveland Press, 2013.
- [4] R. M. Tomasulo, “An efficient algorithm for exploiting multiple arithmetic units,” *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, Jan. 1967.
- [5] J. E. Smith and A. R. Pleszkun, “Implementing precise interrupts in pipelined processors,” *IEEE Transactions on Computers*, vol. 37, no. 5, pp. 562–573, May 1988.
- [6] S. McFarling, “Combining branch predictors,” Digital Western Research Laboratory, Palo Alto, CA, USA, Tech. Rep. TN-36, 1993.
- [7] M. Jørgensen, “Ripes: A visual computer architecture simulator and assembly code editor for the RISC-V ISA,” 2021. [Online]. Available: <https://github.com/mortbopet/Ripes>
- [8] N. Binkert *et al.*, “The gem5 simulator,” *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, Aug. 2011. [Online]. Available: <https://www.gem5.org>

11 Apêndices

11.1 Apêndice I — Sequências de Instruções Utilizadas

Abaixo encontram-se os blocos de código completos que serviram de insumo para as rotinas experimentais deste relatório.

11.2 Apêndice II — Dados Brutos e Repositório Git

Os dados completos extraídos das rodadas de simulação, bem como os *scripts* desenvolvidos para a plotagem dos gráficos de desempenho, encontram-se versionados de forma pública no repositório acessível pelo link: <https://github.com/ArthurViel/Trabalho-ACIII>.